

CO_3 ASSESSMENT_2 – CASE STUDY

CSA0609 - Design and Analysis of Algorithms

NAME: Jeeva

REG NO: 192421334

DIGITAL MUSIC LIBRARY SORTING USING MERGE SORT

INTRODUCTION

In the modern digital era, music streaming services have become one of the most widely used online platforms. Applications such as Spotify, Apple Music, YouTube Music, Amazon Music, and JioSaavn manage millions of songs that are accessed by users across the world every day. These platforms continuously add new songs, albums, podcasts, and playlists to their databases. As the collection grows, organizing and retrieving songs efficiently becomes a major challenge. When users search for songs or browse playlists, they expect the results to appear instantly and in the correct order. Songs may need to be arranged based on release date, popularity, artist name, album name, genre, or duration. If the sorting process is slow or inaccurate, users may experience delays, inconsistent playlists, and poor search results, reducing the overall quality of the platform.

Sorting algorithms play an important role in organizing large collections of music. They ensure that songs are arranged systematically, making searching, filtering, and playlist generation faster and more efficient. Choosing the correct sorting algorithm is essential because the performance of the entire music platform depends on how quickly and accurately data can be processed. Among the various sorting algorithms available, Merge Sort is considered one of the most reliable algorithms for handling large datasets. It uses the divide-and-conquer technique, where a large dataset is divided into smaller parts, each part is sorted individually, and finally all parts are merged together to produce a completely sorted list. This approach provides consistent performance regardless of whether the input data is already sorted, partially sorted, or completely random.

DESCRIPTION OF THE REAL-WORLD PROBLEM

Digital music streaming platforms maintain enormous collections of audio files. Every day, thousands of new songs are uploaded by artists from different countries and languages. The database may contain millions of records, including song title, artist name, album, release year, popularity score, genre, language, and duration. Users interact with these platforms in different ways. Some users search for the latest songs, while others prefer the most popular songs or songs by a particular artist. Playlists are also automatically generated based on user preferences and listening history. All these operations require the music library to be sorted efficiently. Suppose a music platform wants to display all songs released in chronological order. The platform must compare the release year of every song and arrange them from the

oldest to the newest. If two songs were released in the same year, their original upload order should remain unchanged to maintain consistency. This requirement makes stable sorting extremely important.

Handling such a large volume of data creates several challenges:

- The number of songs increases continuously.
- Sorting should be completed within a very short time.
- The algorithm should provide consistent performance for every dataset.
- Duplicate release dates should not disturb the original ordering.
- The system should be capable of processing data stored on external storage devices.

Traditional sorting methods such as Bubble Sort or Selection Sort require a large number of comparisons when the dataset becomes very large. Their quadratic time complexity makes them unsuitable for digital music libraries containing millions of songs. The sorting process may become slow and consume unnecessary computational resources. Merge Sort addresses these challenges by dividing the dataset into smaller subproblems. Each smaller list is sorted independently and later merged into a completely sorted list.

PROBLEM STATEMENT

Modern music streaming applications require an efficient method to organize millions of songs according to release date while maintaining consistency among songs having identical release years. The existing challenge is to perform sorting quickly without affecting playlist order or increasing processing time as the dataset grows.

The system should be capable of handling continuously expanding music libraries while providing accurate and stable sorting results. It must minimize execution time, support largescale datasets, and ensure that songs with equal release dates preserve their original sequence. An efficient algorithm is therefore required to improve search performance, playlist generation, recommendation systems, and overall user experience.

The objective of this case study is to analyse how Merge Sort can be applied to organize a digital music library efficiently using the divide-and-conquer approach. The study also evaluates the algorithm's time complexity, stability, scalability, advantages, limitations, and suitability for large real-world datasets.

ALGORITHM USED – MERGE SORT

Merge Sort is a divide-and-conquer sorting algorithm used to arrange a large collection of data in a specific order. In the digital music library, Merge Sort is used to sort songs based on their release year or other attributes such as popularity, artist name, or song title.

The algorithm works in three main steps:

1. Divide: The list of songs is divided into two smaller halves.
2. Conquer: Each half is recursively sorted using Merge Sort.
3. Merge: The two sorted halves are compared and combined to form one completely sorted list.

For example:

Original:

2021 2019 2023 2020 2022

Divide:

2021 2019 2023 | 2020 2022 Sort

and Merge:

2019 2021 2023 | 2020 2022

Final:

2019 2020 2021 2022 2023

Merge Sort continues this process until the complete music library is sorted.

JUSTIFICATION FOR CHOOSING THE ALGORITHM

Merge Sort is selected for the digital music library because it is efficient, stable, and suitable for handling large datasets containing millions of songs.

1. **Efficient for Large Data** - A music platform may contain millions of songs. Merge Sort has a time complexity of $O(n \log n)$, making it much faster than algorithms such as Bubble Sort and Selection Sort for large datasets.
2. **Stable Sorting** - Merge Sort is a stable sorting algorithm. If two songs have the same release year, their original order is preserved. This is important for maintaining consistency in playlists and music records.
3. **Guaranteed Performance** - Merge Sort provides $O(n \log n)$ time complexity in the best, average, and worst cases. Therefore, its performance remains predictable even when the songs are already sorted or arranged randomly.
4. **Divide-and-Conquer Approach** - The algorithm divides a large music library into smaller sections, sorts them independently, and combines them. This reduces the complexity of processing a very large collection.
5. **Suitable for External Sorting** - If the music database is too large to fit into main memory, Merge Sort can process data stored in external files. This makes it suitable for large-scale database and cloud-based music platforms.

6. Consistent Results - Merge Sort produces reliable and consistent sorting results regardless of the initial arrangement of songs. This helps maintain accurate music catalogs and improves the user experience.

COMPLEXITY ANALYSIS

Merge Sort is an efficient sorting algorithm for organizing a large digital music library. It follows the divide-and-conquer technique, where the complete list of songs is repeatedly divided into smaller sub lists. After sorting the smaller sub lists, they are merged to produce the final sorted list.

CASE	TIME COMPLEXITY	EXPLANATION
Best Case	$O(n \log n)$	The data is divided and merged even when the songs are already sorted.
Average Case	$O(n \log n)$	The algorithm provides efficient performance for normally arranged data.
Worst Case	$O(n \log n)$	The performance remains the same even when the songs are in reverse order.
Space Complexity	$O(n)$	Extra memory is required for temporary arrays during merging.

The time complexity of Merge Sort is $O(n \log n)$ because the dataset is divided into two parts at every stage. The division process creates approximately $\log_2 n$ levels. At each level, all the songs are processed during the merging operation, which requires approximately $O(n)$ time. Therefore, the total complexity becomes $O(n \log n)$. For example, if a digital music platform contains 1,024 songs, the data can be divided into smaller groups for approximately 10 levels because $\log_2(1024) = 10$. This demonstrates why Merge Sort can efficiently handle a large number of songs. The additional space complexity is $O(n)$ because temporary arrays are required while combining the sorted sublists. Although additional memory is required, the algorithm provides reliable performance and is suitable for large-scale music databases.

MANUAL EXECUTION (STEP-BY-STEP PROCESS)

Consider a digital music library containing five songs. The songs are initially arranged randomly according to their release year.

SONG	RELEASE YEAR
------	--------------

Song A	2021
Song B	2019
SONG	RELEASE YEAR
Song C	2023
Song D	2020
Song E	2022

The objective is to arrange these songs in ascending order of release year using Merge Sort.

Step 1: Initial List

The original list is:

2021 2019 2023 2020 2022

At this stage, the songs are not arranged in chronological order. Merge Sort begins by dividing the complete list into smaller parts.

Step 2: First Division

The list is divided into two sections:

2021 2019 2023 | 2020 2022

The first section contains three elements, while the second section contains two elements. The algorithm continues dividing these sections until individual elements are obtained.

Step 3: Second Division

The first section is divided again:

2021 | 2019 2023

The second section is divided as:

2020 | 2022

The remaining group containing 2019 and 2023 is divided further: 2019

| 2023

Now every sublist contains only one element.

Step 4: First Merge Operation

The values **2019** and **2023** are compared. Since 2019 is smaller than 2023, 2019 is placed first.

2019 2023

Next, 2021 is compared with the sorted values.

The resulting sorted group is:

2019 2021 2023

The other two elements are already individually sorted:

2020 2022

Step 5: Final Merge Operation

The two sorted groups are now:

First Group: 2019 2021 2023

Second Group: 2020 2022

The algorithm compares the first elements of both groups.

- Compare **2019** and **2020** → 2019 is smaller.
- Compare **2021** and **2020** → 2020 is smaller.
- Compare **2021** and **2022** → 2021 is smaller.
- Compare **2023** and **2022** → 2022 is smaller.
- The remaining value **2023** is added at the end.

Therefore, the final sorted list is:

2019 2020 2021 2022 2023

Final Result

POSITION	SONG	RELEASE YEAR
1	Song B	2019
2	Song D	2020
3	Song A	2021

4	Song E	2022
5	Song C	2023

The manual execution demonstrates how Merge Sort divides a large problem into smaller problems and then combines the results systematically. This approach avoids unnecessary comparisons and provides efficient sorting. In a real digital music library containing millions of songs, the same process is performed recursively on much larger datasets.

Since Merge Sort is stable, if two songs have the same release year, the song that appeared first in the original library will remain before the other song after sorting. This makes the algorithm useful for maintaining consistency and predictable ordering in digital music platforms.

APPLICATIONS

Merge Sort has several practical applications in digital music libraries and other large-scale data management systems.

1. **Digital Music Platforms:** Merge Sort can arrange millions of songs based on release date, popularity, song title, artist name, or album name.
2. **Playlist Management:** It can organize songs in chronological or alphabetical order while maintaining the original order of songs having the same sorting value.
3. **Music Database Management:** Large databases containing song information can use Merge Sort to efficiently organize records for faster searching and retrieval.
4. **External File Sorting:** When the music collection is too large to fit into main memory, Merge Sort can sort data stored in external files or storage devices.
5. **Cloud-Based Music Systems:** Cloud platforms can use Merge Sort for processing and organizing large collections of music records.
6. **Search and Filtering:** Sorted song records make it easier to search and filter songs based on release year, artist, popularity, or other attributes.
7. **Recommendation Systems:** Properly sorted music data can support recommendation systems by organizing songs according to popularity, ratings, release date, or listening frequency.

ADVANTAGES

1. **Efficient for Large Datasets:** Merge Sort performs well when the number of songs is very large. Its $O(n \log n)$ complexity makes it suitable for millions of records.

2. **Stable Sorting:** It preserves the original order of songs with equal values. For example, songs released in the same year remain in their original sequence.
3. **Guaranteed Performance:** Merge Sort has $O(n \log n)$ time complexity in the best, average, and worst cases.
4. **Consistent Results:** The algorithm provides predictable performance regardless of whether the input data is sorted, randomly arranged, or in reverse order.
5. **Suitable for External Sorting:** It can efficiently process data stored in external files when the entire dataset cannot fit into memory.
6. **Divide-and-Conquer Approach:** Breaking a large dataset into smaller parts makes the sorting process easier to manage and understand.
7. **Scalable:** Merge Sort can handle increasing amounts of data without a significant change in its performance pattern.

LIMITATIONS

1. **Extra Memory Requirement:** Merge Sort requires $O(n)$ additional memory for temporary arrays during the merging process.
2. **Not In-Place:** Unlike some sorting algorithms, the standard Merge Sort implementation does not sort the entire dataset within the original memory space.
3. **Recursive Overhead:** The recursive calls used to divide the dataset may create additional function-call overhead.
4. **Slower for Small Datasets:** For small collections of songs, simpler algorithms such as Insertion Sort may sometimes be faster because of lower overhead.
5. **Memory Management:** When extremely large datasets are processed, the additional memory required during merging can increase system resource usage.
6. **Implementation Complexity:** Merge Sort is slightly more complicated to implement than simple algorithms such as Bubble Sort or Selection Sort.
7. **Data Copying:** During the merge operation, elements may need to be copied into temporary arrays, which can increase memory access and processing overhead.

Comparison with Other Algorithms

Sorting algorithms differ in their execution time, memory requirements, stability, and suitability for large datasets. For a digital music library containing millions of songs, the algorithm should provide fast and consistent performance. The following table compares Merge Sort with other commonly used sorting algorithms.

Algorithm	Best Case	Average Case	Worst Case	Space	Stable	Large Dataset
-----------	-----------	--------------	------------	-------	--------	---------------

Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Not Suitable
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Not Suitable
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Limited
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Suitable
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Suitable
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Highly Suitable

COMPARISON AND ANALYSIS

1. **Bubble Sort** is simple to implement but has $O(n^2)$ average and worst-case complexity. Therefore, it is not appropriate for a music platform containing millions of songs.
2. **Selection Sort** also requires $O(n^2)$ comparisons. Although it uses very little additional memory, its performance is poor for large datasets.
3. **Insertion Sort** performs well for small or nearly sorted datasets, but its $O(n^2)$ average and worst-case complexity makes it unsuitable for very large music libraries.
4. **Quick Sort** is generally very fast and has $O(n \log n)$ average-case performance. However, its worst-case complexity can become $O(n^2)$, and it is not stable in its standard implementation.
5. **Heap Sort** provides $O(n \log n)$ worst-case performance and uses limited additional memory. However, it is not stable, which makes it less suitable when maintaining the original order of songs with equal release years is important.
6. **Merge Sort** provides $O(n \log n)$ performance in all cases and is stable. Therefore, it is a strong choice for a digital music library where millions of songs must be sorted efficiently and consistently.

SCREENSHOT OF THE PROGRAM

```
1- def merge_sort(arr):
2-     if len(arr) > 1:
3-         mid = len(arr) // 2
4-         left = arr[:mid]
5-         right = arr[mid:]
6-         merge_sort(left)
7-         merge_sort(right)
8-         i = j = k = 0
9-         while i < len(left) and j < len(right):
10-            if left[i][1] <= right[j][1]:
11-                arr[k] = left[i]
12-                i += 1
13-            else:
14-                arr[k] = right[j]
15-                j += 1
16-            k += 1
17-         while i < len(left):
18-             arr[k] = left[i]
19-             i += 1
20-             k += 1
21-         while j < len(right):
22-             arr[k] = right[j]
23-             j += 1
24-             k += 1
25- n = int(input("Enter the number of songs: "))
26- songs = []
27- for i in range(n):
28-     print(f"\nEnter details of Song {i + 1}")
29-     name = input("Song Name: ")
30-     year = int(input("Release Year: "))
31-     songs.append((name, year))
32- print("\nMusic Library Before Sorting")
33- print("-----")
34- for song in songs:
35-     print(f"{song[0]} - {song[1]}")
36- merge_sort(songs)
37- print("\nMusic Library After Sorting (Release Year)")
38- print("-----")
39- for song in songs:
40-     print(f"{song[0]} - {song[1]}")
```

OUTPUT SCREENSHOT

```
Music Library Before Sorting
-----
bliever - 2017
shape of you - 2017
perfect - 2018
Faded - 2015
Blinding Lights - 2019

Music Library After Sorting (Release Year)
-----
Faded - 2015
bliever - 2017
shape of you - 2017
perfect - 2018
Blinding Lights - 2019
```

CONCLUSION

Digital music platforms manage millions of songs and require efficient sorting techniques to organize their large collections. Songs may need to be sorted according to release date, popularity, artist name, album, or other attributes. As the size of the music library increases, selecting an appropriate sorting algorithm becomes important for maintaining fast and reliable system performance. In this case study, Merge Sort was used to organize songs according to their release year. The algorithm follows the divide-and-conquer approach by repeatedly dividing the music collection into smaller sublists, sorting those sublists, and finally merging them to produce the complete sorted collection.

The analysis shows that Merge Sort has a time complexity of $O(n \log n)$ in the best, average, and worst cases. This predictable performance makes it suitable for large music databases. Another important advantage is its stability, which preserves the original order of songs having the same release year. This helps maintain consistency in playlists and music records. Compared with Bubble Sort, Selection Sort, and Insertion Sort, Merge Sort is considerably more efficient for large datasets. Although Quick Sort and Heap Sort also provide good performance, Merge Sort has the additional advantage of stable sorting. Its main limitation is the additional $O(n)$ memory required during the merging process. Therefore, Merge Sort is an effective and reliable solution for sorting large digital music libraries. Its efficiency, stability, scalability, and predictable performance make it suitable for modern music streaming applications and other large-scale data management systems.